Weighted Token Swapping on Graphs

project report submitted to the Amrita Vishwa Vidyapeetham in partial fulfilment of the requirement for the Degree of

BACHELOR OF COMPUTER APPLICATIONS

Submitted by

Registration Number	Name of the student
AM.EN.U3BCA22001	ABHIJITH V
AM.EN.U3BCA22049	SACHIN S
AM.EN.U3BCA22053	SHREERAM V RAKESH
AM.EN.U3BCA22057	THARUN BABU

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

AMRITA SCHOOL OF COMPUTING

AMRITA VISHWA VIDYAPEETHAM AMRITAPURI CAMPUS

June 2025

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

AMRITA VISHWA VIDYAPEETHAM AMRITAPURI CAMPUS

BONAFIDE CERTIFICATE

This is to certify that ABHIJITH V , SACHIN S , SHREERAM V RAKESH , and THARUN BABU (bearing Roll Numbers AM.EN.U3BCA22001, AM.EN.U3BCA22049, AM.EN.U3BCA22053, and AM.EN.U3BCA22057 respectively), have successfully completed their undergraduate project titled:

"Weighted Token Swapping on Graphs"

in partial fulfillment of the requirements for the award of the Bachelor of Computer Applications (BCA) degree at the Amrita School of Computing, Amrita Vishwa Vidyapeetham, during the academic year 2024-2025, under my guidance and supervision.

I certify that these students has fulfilled all the requirements necessary for the completion of the project and is eligible for the Bachelor of Computer Applications award. I wish all the success in their future endeavours.

Signature of Project Guide with Date
Dr. Indulekha T S
Department of Computer Science and Applications

Signature of Project Coordinator with Date
Ms. Anu Prabha R S
Department of Computer Science and Applications

The Project was evaluated by us on .....

INTERNAL EXAMINER EXTERNAL EXAMINER

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS
AMRITA VISHWA VIDYAPEETHAM AMRITAPURI CAMPUS

DECLARATION

We, ABHIJITH V, SACHIN S, SHREERAM V RAKESH, and THARUN BABU (AM.EN.U3BCA22001, AM.EN.U3BCA22049, AM.EN.U3BCA22053 and AM.EN.U3BCA22057), hereby declare that this project entitled **Weighted Token Swapping on Graph** is a record of the original work done by us under the guidance of Dr. Indulekha T S, Amrita School of Computing, Amrita Vishwa Vidyapeetham that this work has not formed the basis for any degree/diploma/associationship/fellowship or similar awards to any candidate in any university to the best of our knowledge.

Signature of Student(s) Guide Name with Signature

Place: Amritapuri Campus

Date:

Acknowledgements

We would like to express our sincere gratitude to all those who have helped us in completing this project successfully. This project has given us the opportunity to enhance our skills and knowledge in the field of computer applications.

First and foremost, we would like to acknowledge Sri. Mata Amritanandamayi Devi, our Chancellor and guiding light, for giving us the knowledge and opportunity to undertake this project satisfactorily

We want to thank our project guide, Dr. Indulekha T S, for her constant support and guidance throughout the project. Her valuable suggestions and feedback have been instrumental in shaping this project.

We are also grateful to Ms. Anu Prabha R S, Faculty Associate, for providing us with the necessary resources and infrastructure to carry out this project. We also thank the management of Amrita Vishwa Vidyapeetham for allowing us to work on this project.

Last but not least, we would like to acknowledge the contributions of our fellow team members who have worked alongside us on this project. Their dedication and teamwork have been the key to the success of this project.

ABHIJITH V
SACHIN S
SHREERAM V RAKESH
THARUN BABU

Abstract

Token swapping is the task of rearranging tokens on a graph so that each reaches its designated position using a sequence of swaps. In the weighted variant, the cost of a swap equals the sum of the weights of the two tokens involved. The algorithms for structures like paths and stars exist, the problem becomes more complex on general trees. In this project, we implement and evaluate algorithms for weighted token swapping on paths and stars, and extend our study to the broom topology—a hybrid of a path and a star. Our work includes implementations, test cases, and observations that highlight the potential for efficient strategies on brooms and beyond.

Contents

1 Introduction

Token swapping is a combinatorial optimization problem where tokens on a graph must be rearranged to their correct destinations using a sequence of adjacent swaps. This problem arises in domains like robotics, network routing, and genome sequencing. In the unweighted version, the number of swaps is minimized, while in the weighted version, each token carries a weight and the cost of a swap is the sum of the weights of the swapped tokens.

Previous studies have solved the unweighted token swapping problem on graphs such as paths, stars, and brooms with polynomial-time algorithms. However, the general version of the problem is NP-hard, and the weighted variant introduces further complexity. In this work, we explore weighted token swapping on broom graphs, which combine star and path structures, introducing new challenges. Our goal is to adapt and extend known methods to develop efficient strategies specific to this hybrid structure.

Sorting permutations with various operations has applications in genetics and computer interconnection networks. The alphabet we employ for permutations is 1, 2, $\cdots$, n - 1, n to generate the Symmetric group Sn. The permutation I = (1, 2, 3, 4, $\cdots$, n - 1, n) is the identity permutation in S_n. An operation G is specified by a generator set of Sn. $G \subset S_n$. Each permutation in the generator set is a generator. Let π be a permutation in S_n, and σ be a generator. Applying σ on π yields another permutation, say $\gamma \in S_n$. We say that σ 'transforms' π into γ. Given two permutations α, $\beta \in S_n$, and an operation G, the minimum number of generators of G that are required to transform α into β is called the distance from α to β under G, denoted by dG(α, β). When G is symmetric, dG(α, β) = dG(β, α). If the target permutation is I, then dG(α, I) is succinctly denoted as dG(α), which is called the distance of α. Transforming α into I is called as sorting, and thus dG(α) is the number of generators required to sort α parsimoniously. Let g1, g2, . . . gt be a shortest sequence of generators such that $\alpha \circ$ g1 $\circ$ g2 . . . gt = β. Then β -1 $\circ\alpha\circ$g1 $\circ$g2 . . . gt = β -1 $\circ\beta$ = I. That is, the number of moves required to transform α into β is same as that of transforming β -1 $\circ$ α into the identity permutation I. Thus, the problem of finding minimum distance between two given permutations reduces to the problem of sorting a related permutation. If the target permutation is I, then dG(β -1 $\circ$ α, I) is

succinctly denoted as dG(β -1 $\circ$ α), which is called the distance of a single permutation β -1 $\circ$ α. We study the problem of sorting permutations under the operation specified by a transposition tree.

An interconnection network is a network of interconnected devices and refers to the network used to route data between the processors in a multiprocessor computing system. The interconnection network is typically modeled as a graph. The vertices of the graph correspond to processors, and two vertices are adjacent in the graph whenever there is a direct communication link between the two corresponding processors. One of the metrics to measure the performance of an interconnection network is its diameter. The diameter of a graph is defined as the maximum distance between two vertices of the graph. The maximum latency between any pair of nodes in computer interconnection networks is determined by the diameter of the underlying Cayley graph (corresponding to the given operation and alphabet). It was shown that the diameter of some families of Cayley graphs is sublogarithmic in the number of vertices [2]. This is one of the key reasons why Cayley graphs were chosen as the topology of interconnection networks instead of hypercubes.

The design and performance of algorithms or bounds that determine or estimate the diameter of various families of Cayley graphs of permutation groups is of much theoretical and practical interest. The problem of determining the diameter of a Cayley network is the same as that of determining the diameter of the corresponding group for a given set of generators; the latter quantity is defined to be the minimum length of an expression for a group element in terms of the generators. Each transposition tree is an operation with the form T = (V, E), where E is the generating set that is closed under inverses.

In the remainder of the article, 'sorting' refers to transforming a given permutation into the identity permutation with minimum number of moves of T. The focus of this thesis is on calculating the upper bound for sorting permutations using transposition trees and designing optimal algorithms for sorting permutations using various Transposition tress.Here we focus on the transposition trees such as path , star, single broom and double broom. Each of these structures has unique properties that have a bearing on the number of moves required to sort as well as the complexity of the algorithm associated with it.

2 Problem Definition

The Weighted Token Swapping Problem is a generalization of the classical token swapping problem, where each token has an associated weight, and the cost of a swap is defined as the sum of the weights of the two swapped tokens. The objective is to move each token to its target position using a sequence of swaps with minimum total cost.

Formally, given a graph $G = (V, E)$ with n vertices and a set of tokens $T = [t_1, t_2, ...t_n]$ where each token t_i is assigned a weight $w(t_i) \in Z^+$, the task is to compute a sequence of adjacent swaps such that all tokens reach their correct positions, and the sum of the costs of all swaps is minimized.

This problem is known to be NP-hard on general graphs and even on trees when extended to the weighted setting. The complexity arises from the interaction between token positions, graph structure, and the weights assigned to tokens. Due to the absence of a efficient solution, we approach the problem by experimenting with simplified scenarios. Specifically, we assign weights of either 0 or 1 to the tokens and evaluate various scenarios by changing the placement of these weights. This method allows us to study how different weight assignments affect the total cost and to explore possible heuristic strategies for specific tree structures.

Scope of this Paper

In this paper, we focus on a specialized variation known as Weighted Token Swapping on Broom Graphs, where the underlying topology is a combination of a path and a star. Each token $t_i \in T$ is associated with a positive integer weight $w(t_i)$. The cost of swapping two tokens t_i and t_j is defined as $w(t_i) + w(t_j)$. Our objective is to identify a valid sequence of swaps that repositions all tokens to their respective target vertices with the least possible total cost.

By constructing and analyzing different weighted configurations on the broom structure, we aim to:

- Observe the impact of weight distribution on swap cost
- Identify cost patterns unique to broom graphs,
- Evaluate the complexity and behavior of weighted token swapping in a hybrid tree topology.

3 Related Work

The token swapping problem has been widely explored on tree-based structures due to its applications in sorting, interconnection networks, and robotics. Efficient solutions exist for simpler topologies like paths and stars, where sorting strategies rely on inversion counting and cycle decomposition. However, hybrid structures such as brooms—formed by combining a path and a star—present additional challenges.

The thesis *"Optimal Algorithms for Sorting Permutations with Brooms"* by Indulekha T S introduces optimal algorithms for unweighted token swapping on single brooms. The work provides structured approaches to minimize swaps by separating path and star elements and applying targeted strategies.The vertices in a single broom are partitioned into star vertices and path vertices, with corresponding star and path markers that need to be homed to their respective positions.

This thesis introduces Algorithm Ab for sorting permutations on a single broom, which operates in three phases: it first homes the farthest unhomed star marker on the path (S_{min}), then homes the highest unhomed path marker (P_{max}), and finally homes the remaining star markers on the star. The algorithm runs in $O(n^2)$ time and is proven to be optimal.

4 Background Knowledge

4.1 Group

A group is an algebraic structure consisting of a set of elements equipped with an operation that combines any two elements to form a third element and that satisfies four conditions called the group axioms, namely closure, associativity, identity, and invertibility.

4.2 Permutation Group

A permutation group is a group G whose elements are permutations of a given set M and whose group operation is the composition of permutations in G (which are thought of as bijective functions from the set M to itself). The group of all permutations of a set M is the symmetric group of M, often written as Sym(M). The term permutation group thus means a subgroup of the symmetric group. If M = 1, 2, . . . , n, then Sym(M), the symmetric group on n letters, is usually denoted by Sn. Since permutations are bijections of a set, they can be represented using Cauchy's two-line notation:

$$\sigma = \begin{pmatrix} x_1 & x_2 & x_3 & \cdots & x_n \\ \sigma(x_1) & \sigma(x_2) & \sigma(x_3) & \cdots & \sigma(x_n) \end{pmatrix}$$

This means that σ satisfies $\sigma(1) = 2$, $\sigma(2) = 5$, $\sigma(3) = 4$, $\sigma(4) = 3$, and $\sigma(5) = 1$. The elements of M need not appear in any special order in the 13 first row. This permutation could also be written as:

$$\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & 5 & 4 & 3 & 1 \end{pmatrix}$$

Permutations are also often written in cyclic notation. Given the set M = 1, 2, 3, 4, a permutation σ of M with $\sigma(1) = 2$, $\sigma(2) = 4$, $\sigma(4) = 1$, and $\sigma(3) = 3$ will be written as (1, 2, 4, 3) or more commonly, (1, 2, 4) since 3 is left unchanged. The permutation written above in two-line notation would be written in cyclic notation as:

$$\sigma = \ (1, 2, 3)(3, 4)$$

4.3 Transposition

A cycle with only two elements is called a transposition. For example, the permutation that swaps 2 and 4.

$$\pi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 1 & 4 & 3 & 2 \end{pmatrix} \tag{1}$$

4.4 Transposition trees

Consider a tree on n vertices. We can label the vertices of this tree with the symbols {1, 2, 3, . . . , n} and interpret the edges as transpositions. For example, the tree on six vertices shown in the following figure gives rise to five transpositions: 321456, 132456, 124356, 123546, and 123654. Thus, we can interpret a tree as a set of transpositions, which in turn gives rise to a Cayley graph. We call such a tree, a transposition tree. Just as a set of generators describes a large Cayley graph by exploiting the symmetries of the graph, a transposition tree describes a set of generators by exploiting the symmetries within the generators. Thus, by analysing a tree on n vertices, we can describe properties of a much larger graph, the Cayley graph, on n! vertices.

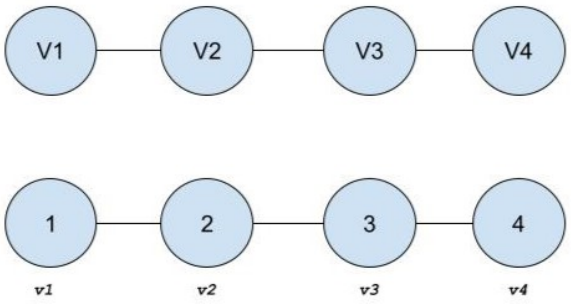
5 Known Result

The token swapping problem has been widely explored in both unweighted and weighted settings across various types of graphs. For unweighted token swapping, efficient polynomial-time algorithms are known for specific graph classes such as paths, stars, and brooms. On paths, the minimum number of swaps corresponds exactly to the number of inversions and can be computed in $O(n \log n)$ time. For star graphs, the optimal swap count is determined by the number of unhappy leaves and the number of non-trivial locked cycles.

Despite these tractable cases, the general token swapping problem remains NP-hard, even in its unweighted version on general graphs . The problem becomes even more complex in the weighted setting, where each token carries a weight, and the cost of a swap is the sum of the weights of the tokens involved. This additional layer of complexity motivates the development of approximation algorithms, particularly for more intricate structures such as broom graphs.

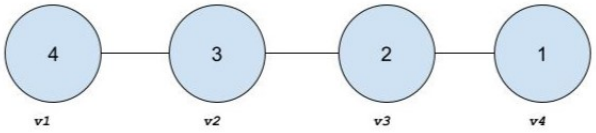
5.1 Line/Path

A line permutation refers to the arrangement of vertex in a sequence, where each vertex is linked to its adjacent vertex, where each vertex Vi is connected to exactly two other vertices, except for the endpoints. Sorting a line permutation involves performing adjacent swaps until the sequence is sorted. In this case, the sorting procedure starts by positioning the largest element in its correct place, followed by the second-largest element, and so on. The sorting process continues step by step, ensuring that at each stage, the largest remaining unsorted element is moved to its correct position.



5.1.1 Worst Case of Path

The worst-case scenario for sorting a line permutation occurs when the permutation is in reverse order, i.e., the largest element appears first, followed by the second-largest, and so on. This is considered the most disordered state, and thus, it requires the maximum number of swaps to sort the elements. For a set of n elements, the reverse permutation requires the maximum number of inversions, which means it will also require the maximum number of swaps. For example,

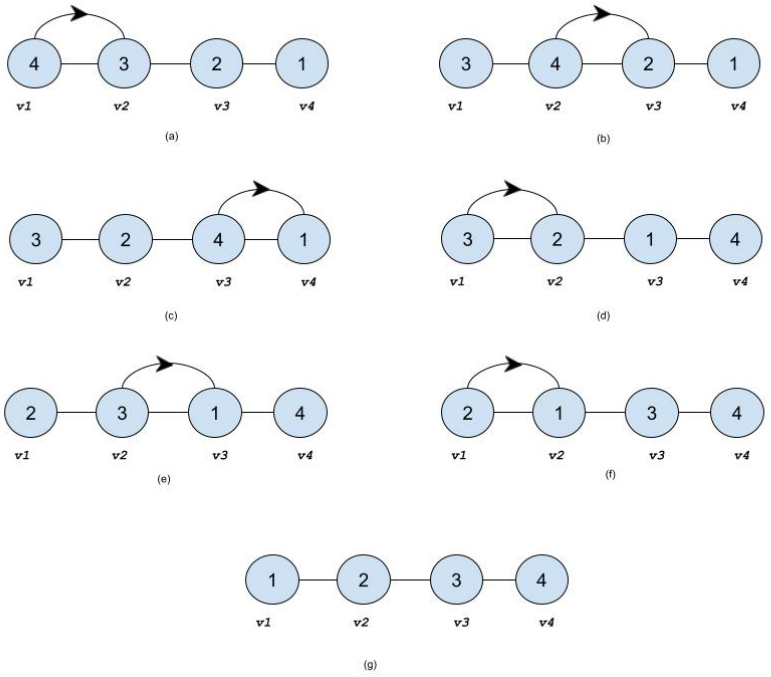


5.1.2 Upper Bound of Line/Path

The upper bound of the number of swaps required to sort a line permutation is determined by the worst-case scenario, where the permutation is in reverse order. In this case, the number of swaps required is the maximum possible number of inversions, which is given by the equation:

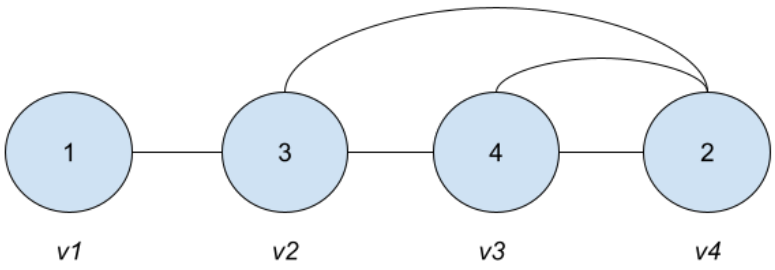
$$UpperBound = \frac{n(n-1)}{2}$$

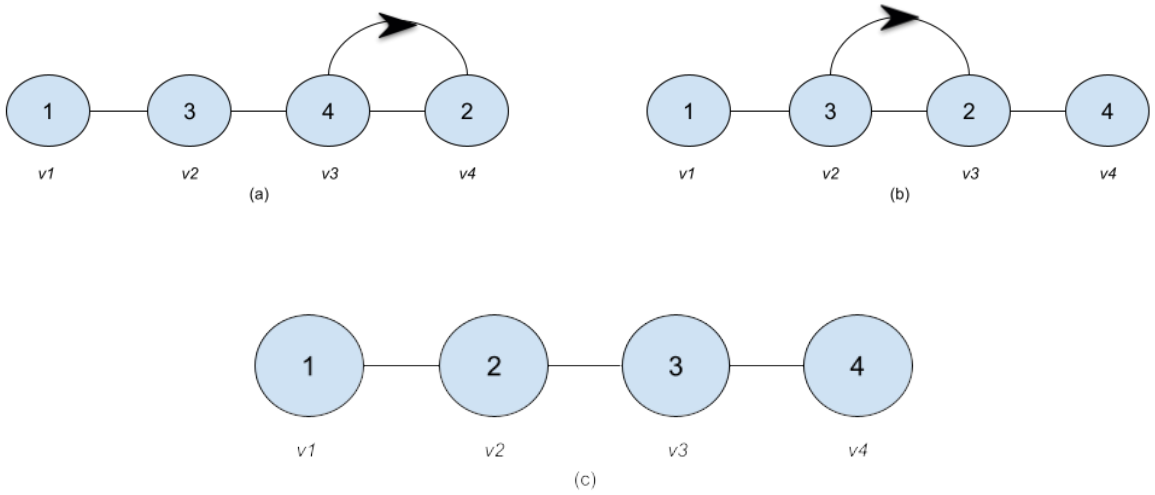
Where n is the number of elements in the permutation. This equation arises because, in the worst-case reverse order, each element needs to be swapped with every other element to achieve the sorted order. For example, if the permutation contains 4 elements, the upper bound is: $\frac{4(4-1)}{2} = 6$. So, for any permutation with 4 elements need maximum 6 swaps.



5.1.3 Optimal of Line/Path

An optimal algorithm for sorting a line permutation minimizes the number of swaps required to sort the sequence. The key for an optimal sorting strategy is that the number of swaps required is equal to the number of inverse pairs in a given permutation. An inverse pair is defined as a pair of elements where the larger element appears before the smaller element in the sequence. For example, in the permutation [1,3,4,2], the inverse pairs are (3,2), (4,2), which gives a total of 2 inverse pairs. The number of inverse pairs directly correlates with the number of swaps required in the optimal sorting algorithm.





5.2 Weighted Tokenswapping on Path

In unweighted case , total cost ie;the total number of swaps is equal to the number of inversions it had. But in the case of Weighted case, The minimum weight of a swap sequence is the sum , overall all inversions t,t'

$$\sum_{inversions(t,t')} [w(t) + w(t')]$$

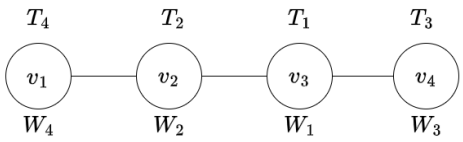


Figure 1: Weighted Token Swapping on Path: Initial Configuration

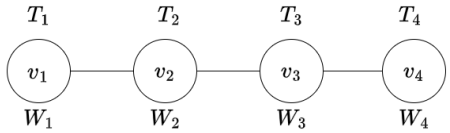
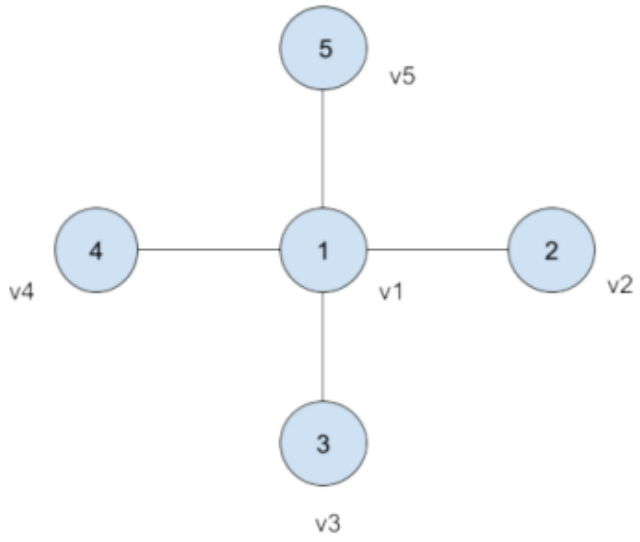


Figure 2: Weighted Token Swapping on Path: Final Configuration

5.3 Star

In a star graph, the structure consists of a central node, V1, and leaf nodes V2, V3, . . . , Vn, each of which is directly connected to V1. This means the edge set of the star graph includes edges of the form (V1, V2), (V1, V3), . . . , (V1, V3). Leaf nodes are not connected to each other; they only have edges linking them to the central node V1. Example: To illustrate, consider the permutation π = [1,2,3,4,5]. Here:

- The first element 1 represents the central node (V1).
- The remaining elements 2,3,4,5 correspond to the leaf nodes (V2, V3, V4, V5).



5.3.1 Optimal Number of Swaps in a Star

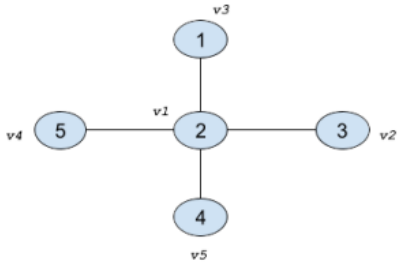
The optimum number of swaps required to sort a permutation in a star graph can be calculated using the formula: *Swaps* = *m*+*c*
where:

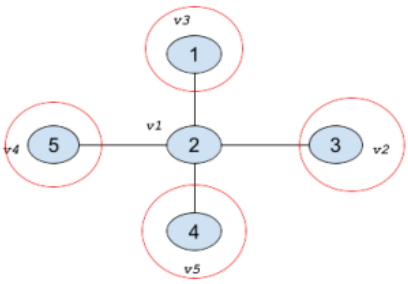
- m = Number of un-homed leaf nodes (leaf nodes not in their correct positions).
- c = Number of cycles in the cycle notation of the permutation. These cycles must satisfy:
  - The cycle length is ≥ 2.
  - Cycles that include the center node (V1) are not counted. Only cycles formed exclusively among the leaf nodes (V2, V3, . . . , Vn) are considered.

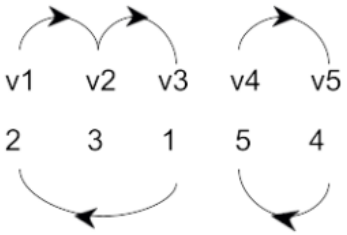
Example:

To illustrate, consider the permutation π = [2,3,1,5,4]. Here:

- The first element 2 represents the central node (V1).
- The remaining elements 3,1,5,4 correspond to the leaf nodes (V2, V3, V4, V5).
- The leaf nodes 3,1,5,4 are not in their correct positions. Thus, m = 4.
- Number of Cycles (c):
  - The cycle includes (1,2,3) the central node V1=2, so it is not counted.
  - The cycle (4,5) is among the leaf nodes only and has a length of 2, so it is counted. Thus, c = 1.







5.3.2 Upper Bound of Swaps in a Star Graph

The upper bound for the number of swaps needed to sort a permutation in a star graph with n elements is given by:

$$UpperBound = \left\lceil \frac{3(n-1)}{2} \right\rceil$$

Explanation:

- The worst-case permutation occurs when the maximum number of disjoint cycles exists among the leaf nodes.
- In a star graph with n elements, there are n-1 leaf nodes. The maximum number

of disjoint cycles in this case is $\frac{n-1}{2}$

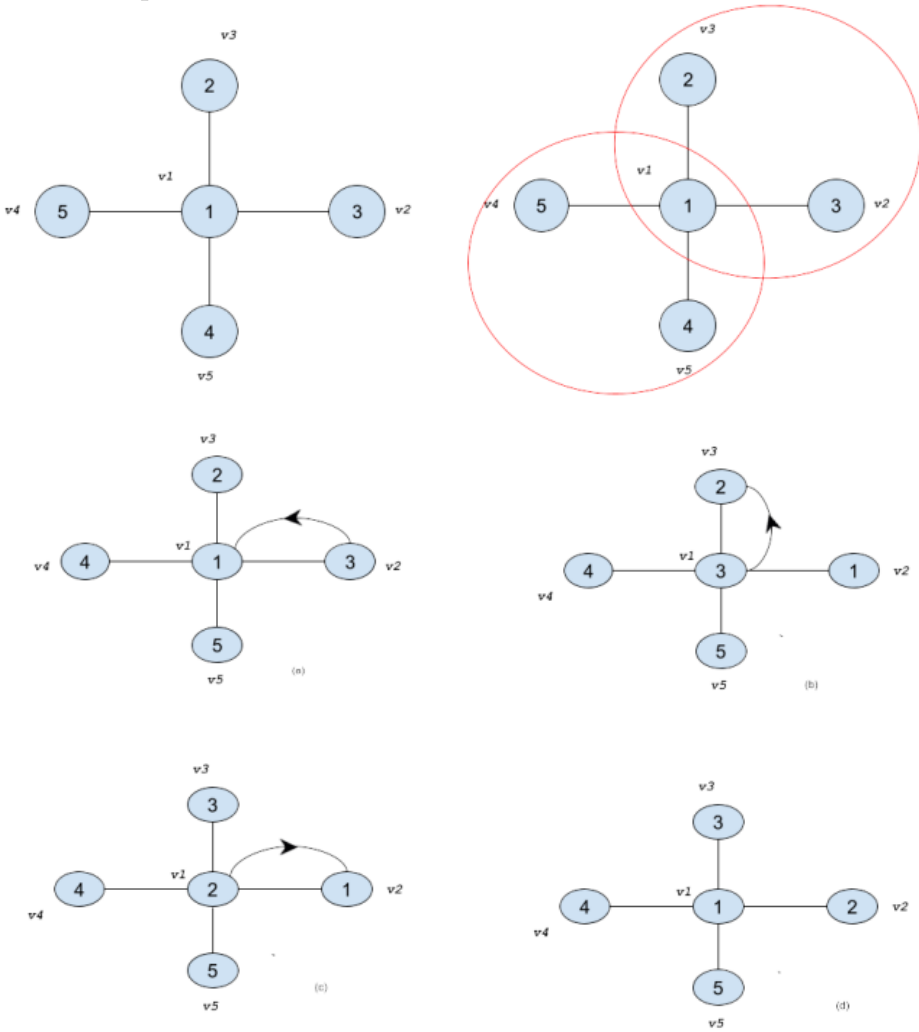
- Sorting a disjoint cycle of length 2 requires exactly 3 swaps in a star graph.
- With a maximum of $\frac{n-1}{2}$ disjoint cycles, and 3 swaps required per cycle, the total number of swaps is

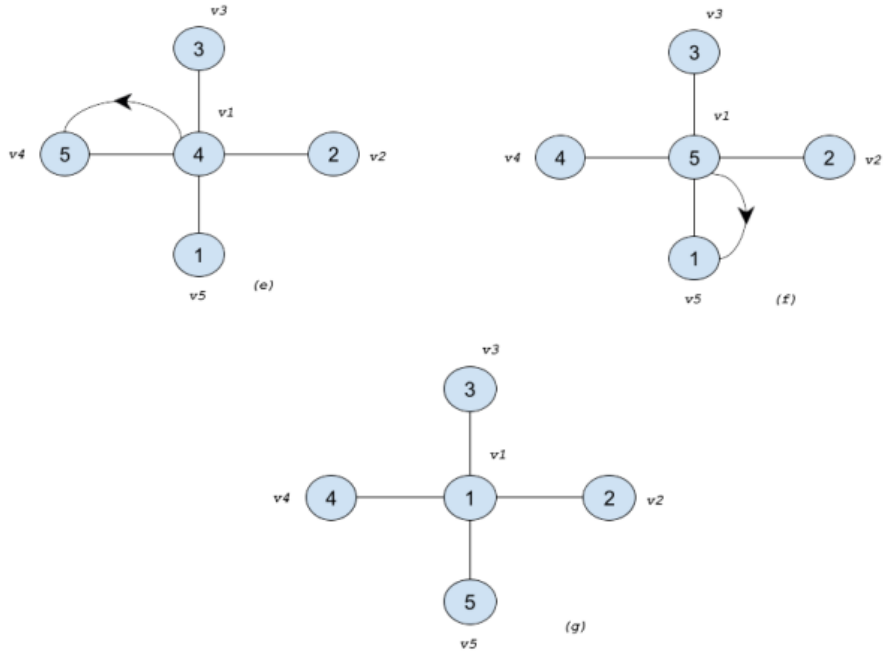
$$3 \times \frac{n-1}{2} = \frac{3(n-1)}{2}$$

Example:

To illustrate, consider the permutation π = [1,3,2,5,4]. Here:

- The first element 1 represents the central node (V1).
- The remaining elements 3,2,5,4 correspond to the leaf nodes (V2, V3, V4, V5)
- The number disjoint cycle: $\frac{n-1}{2} = \frac{5-1}{2} = 2$
- The cycles (3,2) and (5,4) are disjoint cycles among the leaf nodes, each of length 2.
- Swaps=3×2=6





5.4 Weighted Tokenswapping on Star

In weighted token swapping on a star, each token has a distinct weight, allowing us to determine the correct vertex for placement. The swapping process is governed by token weights, where the cost of swapping tokens t and t is given by:

$$w(t) + w(t')$$

The optimal number of swaps in the unweighted case is determined by:

$$n_u + \ell$$

where n_u represents unhappy leaves, and ℓ denotes the number of non-trivial locked cycles. In the weighted case, at least $n_u + \ell$ swaps are needed, but an optimal approach may require up to two additional swaps by leveraging the globally lightest token for cycle resolution.

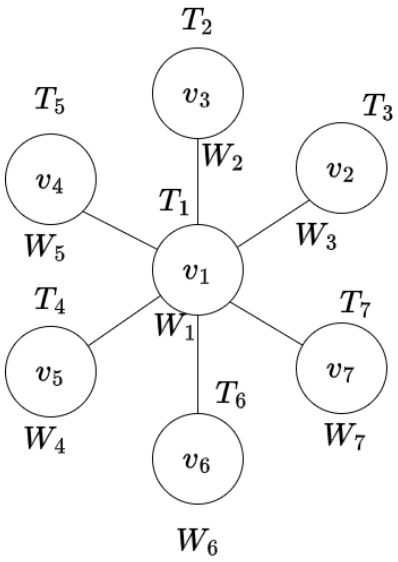


Figure 3: Weighted Token Swapping on Stars: Initial Configuration

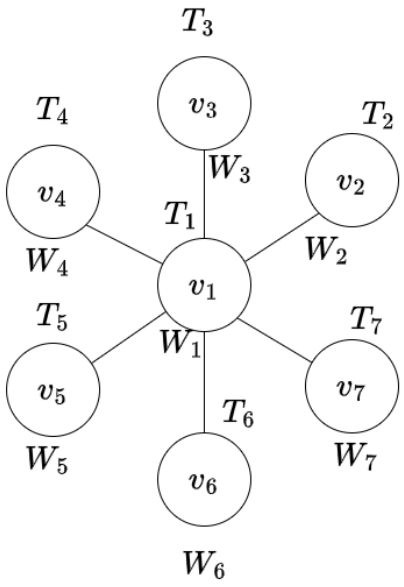


Figure 4: Weighted Token Swapping on Stars: Final Configuration

5.5 Broom

A single broom is a tree obtained by joining the center vertex of a star with one of the two leaf vertices of a path graph, using a new edge. A single broom has n vertices, partitioned into two sets called star vertices v1, v2, . . . , vk, and path vertices vk+1, . . . , vn, where $2 \leq k \leq (n-3)$.

Note that the center of the star, i.e., vk+1 is also a path node. Similarly, markers are

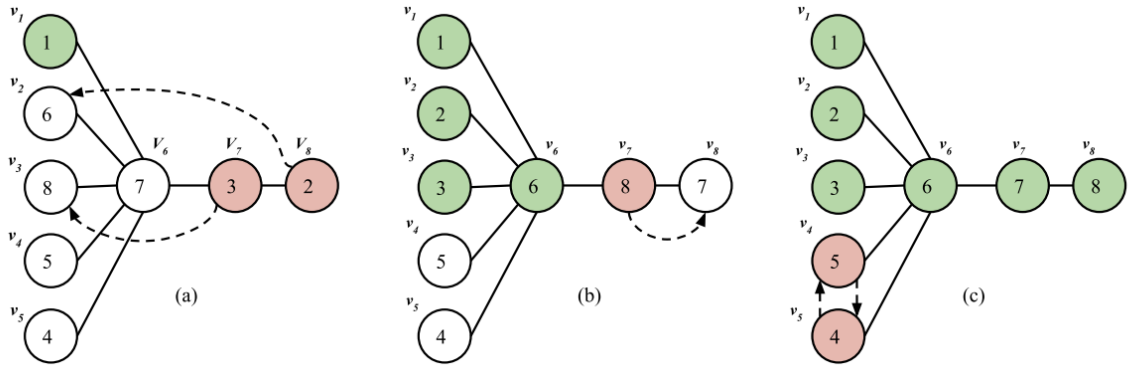
partitioned into two sets called star markers and path markers. A star marker is a marker whose home is a star node, and a path marker is a marker whose home is a path node. An efficient optimal algorithm called A_b, for sorting permutations using single broom is given below.

Algorithm Ab

Let S_{min} be the star marker that is closest to its home residing on the path and P_{max} be the largest path marker that is not homed. Our algorithm A_b for single broom that transforms a given input configuration to a sorted configuration, consists of the following 3 steps in the given order.

1. While ($\exists$ a S_{min}), efficiently home S_{min}
2. While ($\exists$ a P_{max}), efficiently home P_{max}
3. Efficiently home the markers in the star that need to be homed

An illustration of algorithm A_b for a single broom is given in Figure 3.1. In the initial setup, we have two unhomed star markers on path, i.e., 3 on v7 and 2 on v8. S_{min} is 3 in the first iteration and in the next it will be 2. Homing markers 3 and 2 take 5 swaps in total. Then, there will be no star markers residing on the path. Thus, we can start with Step 2 of our algorithm. P_{max} is 8 and a swap with 7 homes both the markers. Only the star markers 5 and 4 at v4 and v5 respectively need to be homed. Our algorithm homes all markers in total of 9 swaps.



Step 1 and Step 2 of A_b have the following properties:

1. Star markers residing on the path always move to the left in the increasing order of their distances from their respective homes.
2. A path marker moves right to its home; thereby moving the smaller path markers towards the center.

3. Swap involving two-star markers will not take place on a path edge.

Anaylsis

Here we implement an algorithm for sorting permutations with a specific emphasis on operations involving a single broom structure for swapping elements optimally. The system sorts a permutation π of length n by dividing it into star markers and path markers. The algorithm proceeds in three key steps:

1. S_{min} Step (Homing Smallest Star Marker):
   - Identifies the star markers closest to its home residing on the path segment ($\pi[k:n]$) that belong to the star segment ($\pi[:k-1]$).
   - These markers are moved efficiently to their home positions in the star segment.

2. P_{max} Step(Homing Maximum Path Marker):
   - Moves the largest unhomed path marker in the path ($\pi[k:n]$)to its home.
   - This step ensures all markers in the path segment are homed

3. Star Swap:
   - Ensures that any misplaced markers in star are homed by swapping and moved to their home.

Here it follows some properties:

1. Star markers residing on the path always move to the left in the increasing order of their distances from their respective homes.
2. A path marker moves right to its home; thereby moving the smaller path markers towards the center.
3. Swap involving two star markers will not take place on a path edge.

Results from Example Input:

Given $\pi = [3, 2, 4, 1]$ with k=2

1. Initial Permutation:

   [3,2,4,1]

   After S_{min} Step:

- Homing star markers 3 and 1 to their positions.
- Swaps: $[(4,3),(2,4)]$
- Result: $[1,2,4,3]$

2. After P_{max} Step:
   - Homing path marker 4.
   - Swaps $[(4,3)]$
   - Result: $[1,2,3,4]$

3. Final Output:

   [1,2,3,4]

Total Swaps:

$$[(4,3),(2,4),(4,3)]$$

Complexity Analysis:

1. Star Marker Homing (S_{min})
   - The algorithm selects the nearest star marker (S_{min}) based on distance and moves it home first.

Let:

- S: Number of star vertices.
- P: Number of path vertices.
- $D(S_p)$: Distance of a star marker S_P from its home.
- W_S: Total number of swaps for homing all SS markers.

The swaps in this step are calculated as:

$$Ws = \sum S_P D(S_P)$$

Worst Case:

- All star markers are at the farthest possible positions (end of the path). The swaps required are:

  $$W_s = P + (P-1) + (P-2) + ... + 1 + (S-P)$$

  Using the formula for the sum of the first P integers:

  $$W_S = \frac{P(P+1)}{2} + (S-P)$$

2. Path Making Homing (P_{max}):

- Path markers are moved to their correct positions using a value-based approach. The algorithm always moves the maximum-valued marker in the path to its home first, ensuring an efficient swap sequence.

Let:

- $D(P_P)$: Distance of a path marker Pp from its home.
- W_P: Total swaps for homing all P markers.

The swaps in this step are calculated as:

$$W_P = \sum_{P_p} D(P_p)$$

Worst Case:

- All P path markers are out of place and require sorting. The total swaps are equivalent to the number of inversions in the permutation:

$$W_P = \frac{P(P-1)}{2}$$

3. Star Cycle Resolution(Star swap):
   - Remaining cycles that involve only star markers (not resolved in Step 1 or 2) are handled here. This includes cycles of length ≥ 2

   Let:

   - L_S: Number of cycles of length ≥ 2 geq 2 involving star markers.
   - N_S:: Number of markers in these cycles.
   - W_{star}: Total swaps in this step.

   The swaps in this step are:

   $$W_{star} = L_S + N_S$$

   Worst Case:

   - All star markers form a single large cycle, requiring the maximum number of swaps:

$$W_{star} = [\frac{3S}{2}]$$

Total Swaps and Worst-Case Time Complexity

The total swaps (W_b) required for sorting a Single Broom is:

$$W_b = W_s + W_p + W_{star}$$

Substituting the worst-case values:

$$W_b = \frac{P(P+1)}{2} + (S-P) + \frac{P(P-1)}{2} + [\frac{3S}{2}]$$

Since $P, S \in O(n)$the worst-case time complexity is:

$$O(n^2)$$

6 Observations and Methodologies

Case 1: Both path and star token misplaced in their on regions

Case 1.1: Path and Star Tokens Misplaced in Their Respective Regions

Case Description:

In this case, all tokens have a weight of 1, except for a single path token $\mathbf{P}$, which has a weight of 0. Token $\mathbf{P}$ is supposed to be homed at index i in the path region. The star region ends at index k, where $k < i$. All tokens, including the light token $\mathbf{P}$, are misplaced but remain within their respective regions—either in the path or star subgraphs.

Strategy

To minimize the overall cost of sorting, the strategy makes use of the zero-weight token $\mathbf{P}$ as a cost-free tool to unlock cycles. If the number of locked cycles l is greater than $2(i - k)$, then moving $\mathbf{P}$ to the central node becomes more beneficial. From there, $\mathbf{P}$ is used to sequentially unlock cycles that would otherwise require swaps involving heavier tokens, incurring more cost. This not only frees up critical positions but also avoids unnecessary use of weight-1 tokens.

Once the cycles are unlocked using $\mathbf{P}$, the farthest misplaced path token (up to index i) is brought to its correct position. After that, $\mathbf{P}$ is finally homed at index i. This order ensures that $\mathbf{P}$ is not prematurely placed before completing its unlocking role.

By dynamically evaluating the trade-off between moving $\mathbf{P}$ and unlocking with heavier tokens, the approach ensures optimal cost-efficiency. The use of the zero-weight token as a utility mover significantly reduces swap cost, making the process both efficient.

Case 1.2: Path and Star Tokens Misplaced — One Star Token with Zero Weight

Case Description:

In this case, all tokens have a weight of 1, except for one token **s** located in the star region, which has a weight of 0. The token **s** is misplaced but still within the star subgraph. The remaining tokens are also misplaced within their respective regions (path or star), and the star region ends at index k.

Strategy

To minimize swap cost, the zero-weight token **s** is strategically brought to the center of the star graph, where it can be used to unlock all locked cycles within the star region. Since **s** incurs no cost in swaps, using it to resolve cycles is highly efficient compared to utilizing heavier tokens. Once the star region is fully resolved using **s**, the path region is addressed next.

In the path segment, the farthest misplaced path token—up to its target index i—is homed using standard path token swapping techniques. This order of operations ensures that **s** fulfills its unlocking role before being placed, allowing the system to resolve high-cost configurations using a cost-free agent. This dynamic strategy ensures optimal management of swap costs by prioritizing light tokens for complex cycle-breaking tasks.

Case 2: Star Token K Positioned in the Path

Case 2.1: K has low weight (0)

If the misplaced star token **K** in the path has weight 0, if there is light weight in the star , it is beneficial to delay homing it immediately. Instead, prioritize using another light (weight-0) star token within the star region to unlock cycles efficiently. Once the unlocking is complete, **K** is then moved back to the star section and eventually homed.If no light weight token is available in star region bring this **K** to centre and use it to unlock the locked cycles.After unlocking home this **K** This case works because the token **K** has weight 0, meaning it can move without increasing the swap cost. Instead of homing **K** immediately, it is used first to unlock cycles in the star region. This helps reduce the total number of costly swaps that would be needed if heavier tokens were used.

After unlocking is done, **K** is moved back to the star and placed in its correct position. By delaying its homing, we take full advantage of its zero cost to make the process more efficient.

Case 2.2: K has heavy weight (1)

When **K** has weight 1, Check if any available light-weight token reside on star region to unlock locked cycles. If no such token is available, Check the closest path token has light weight, if it has it can be brought to the center to assist in unlocking. Only after the unlocking process should **K** be brought to the center and homed to its correct position.If none of these are available bring **k** to its home first.

This approach works because it avoids the high cost associated with early movement of heavy tokens. By prioritizing the use of lighter tokens for unlocking tasks, the overall swap cost is minimized.

Case 3: Two Star Tokens Misplaced in the Path

Case Description:

In this case, two star tokens are incorrectly placed in the path region. The remaining tokens are misplaced within their respective star or path regions. The strategy to resolve this case depends on the weights of the two misplaced star tokens.

Case 3.1: First token is heavy, second is light

In this case, the heavier token is homed early to avoid carrying its cost through multiple swaps. After this, the lighter token is used to unlock cycles in the star region, and then it is homed. Once both tokens are placed correctly, the path region is resolved.

Case 3.2: First token is light, second is heavy

Here, the light token is used first to unlock star-region cycles at minimal cost. Then the heavier token is moved and homed. After that, the remaining misplaced tokens in the path region are resolved.

This approach works effectively because it prioritizes moving the heavy token early when needed and uses the light token to reduce the cost of unlocking. By managing the order of operations based on token weights, the overall swap cost is minimized and efficiency is maintained throughout the process.

Case 4: All Star Tokens on Path, All Path Tokens on Star

Case Description:

In this case, all star tokens are located on the path, and all path tokens are located on the star. This represents a complete region swap between the star and path tokens. The approach to resolving this depends on the weight configuration of the tokens in both regions.

Case 4.1: Star Tokens Weight = 0, Path Tokens Weight = 1

This situation is handled in two phases. In Phase 1, the heavier path tokens (weight 1) are homed first by utilizing the lighter star tokens (weight 0) to assist in unlocking and swapping operations. This avoids heavy-heavy swaps in the early stages. In Phase 2, once the path is resolved, the weight-0 star tokens are placed in their correct positions with minimal or no cost.

This strategy works well because it reduces the use of costly swaps early on and delays the homing of zero-cost tokens to maximize their utility in unlocking operations.

Case 4.2: Star Tokens Random Weight (0 or 1), Path Tokens = 0

Here, Phase 1 focuses on homing all path tokens (which are of weight 0) first. These zero-cost moves free up the star region and the center, which is crucial for efficient routing. In Phase 2, the remaining star tokens are resolved using a cycle-based approach, where low-weight star tokens are used for unlocking when available.

This approach minimizes total cost by leveraging light tokens early, deferring the movement of heavy tokens, and freeing the center node early for efficient swapping. It maintains a low swap cost by aligning the resolution order with token weight and graph structure.

Case 5: Some Star Tokens in Path, Some Path Tokens in Star

Case Description:

In this case, a few star tokens (all with weight 0) are misplaced in the path region, while some path tokens (with random weights) are located in the star region. The configuration represents a partial region swap between star and path tokens.

Strategy and Rationale:

The resolution begins by identifying the heaviest path token currently misplaced in the star region and homing it first. This reduces the impact of carrying costly tokens through multiple swaps. After the heaviest token is homed, the remaining path tokens are resolved, typically using efficient swap sequences that further minimize cost.

Once the path tokens are all homed, the misplaced star tokens (weight 0) are moved back to their respective leaf positions in the star region. Since these tokens have zero weight, they can be moved at no cost and are therefore safely delayed until the final stage.

This strategy is effective because it handles high-cost tokens early, reducing the overall swap weight. By delaying the movement of zero-weight star tokens, their full unlocking potential is preserved and the total cost of sorting remains minimal.

Case 6: All the Tokens in a Region are Homed Except One

Case 6.1: One Star Token Misplaced, All Other Star Tokens Homed

Case Description:

In this case, all star tokens are already homed except for one misplaced star token. Meanwhile, several path tokens are still misplaced. The focus is on resolving the path efficiently while eventually placing the last star token in its correct position.

Strategy

Begin by identifying the farthest misplaced path token. If this token currently resides in the star region (i.e., occupying a leaf of the star), bring it to the center and move it across the path to its correct home position. If the token is already in the path, directly move it to its destination.

Continue this process until all path tokens are correctly placed. At this point, the last misplaced star token will be located at the center node (since it was displaced during the previous steps). Finally, swap it with the token currently occupying its home position in the star leaf.

This strategy works because it avoids unnecessary early movement of the last star token and allows it to be homed efficiently at the end. By focusing first on resolving the more costly and complex movements in the path, and using the center position effectively, the total swap cost is minimized.

Case 6.2: One Path Token Misplaced, All Other Path Tokens Homed

Case Description:

In this case, all path tokens are already in their correct positions except for one misplaced path token. Meanwhile, several star tokens are still misplaced within the star region. The focus is on resolving the star first and then homing the remaining misplaced path token efficiently.

The resolution begins by applying standard star token sorting techniques to home all the

misplaced star tokens. This includes identifying and breaking cycles using low-weight tokens where possible and gradually homing each star token back to its correct leaf position.

Once the star region is fully resolved, the only remaining misplaced token is the path token. If this token is currently located in the star region, it can be brought to the center and moved directly to its home position in the path. If it is already on the path but not homed, a direct movement to its target completes the process.

This approach works well because it focuses on resolving the more complex star token movements first while delaying the placement of the single path token. Since path moves are more straightforward and linear, this order of execution leads to an efficient and low-cost solution.

8. Conclusion

In this project, we set out to understand and solve the Weighted Token Swapping Problem on different types of graphs—starting from simple paths and stars, and moving towards more complex structures like broom graphs. Our main goal was to find the most efficient way to swap tokens so that they reach their correct positions while keeping the total cost of all swaps as low as possible.

We discovered that the placement and weight of tokens play a huge role in determining how costly the swaps can be. Especially interesting was the behavior of zero-weight tokens , which we found to be incredibly useful in unlocking and simplifying difficult configurations. By using them smartly, we were often able to reduce the overall swap cost significantly.

The broom graph, which is a combination of a path and a star, brought unique challenges. To tackle these, we adapted known algorithms and developed strategies that take advantage of the structure of the broom. We handled various real-world-inspired cases where tokens were misplaced in different ways, and by carefully choosing the order in which to move them, we managed to find low-cost solutions.

One of our key takeaways was that timing matters —delaying the placement of a zero-weight token or handling the heaviest token first in some cases helped us minimize the total cost. We also saw how breaking down the problem into smaller, manageable steps—like resolving the path first or unlocking cycles in the star—made the overall task simpler and more effective.

Overall, this project gave us valuable insights into how thoughtful planning and strategic choices can make complex problems easier to solve. While our experiments focused on tokens with weights 0 and 1, the methods and observations we've developed here open up possibilities for exploring more advanced scenarios in the future.

7 References

[1] Ajila, L., and T. S. Indulekha. "Colored Token Swapping Using Broom." 2024 5th IEEE Global Conference for Advancement in Technology (GCAT). IEEE, 2024.

[2] Chitturi, Bhadrachalam, and T. S. Indulekha. "Sorting permutations with a transposition tree." 2019 8th International Conference on Modeling Simulation and Applied Optimization (ICMSAO). IEEE, 2019.

[3] Indulekha, T. S., and Bhadrachalam Chitturi. "Analysis of Algorithm δ^* on full binary trees." 2019 Second International Conference on Advanced Computational and Communication Paradigms (ICACCP). IEEE, 2019.

[4] Sadanandan, Indulekha Thekkethuruthel, and Bhadrachalam Chitturi. "Optimal algorithms for sorting permutations with brooms." Algorithms 15.7 (2022): 220.

[5] Malavika, J., Shijo Scaria, and T. S. Indulekha. "On Token Swapping in Labeled Tree." 2021 Third International Conference on Inventive Research in Computing Applications (ICIRCA). IEEE, 2021.

[6] Yamanaka, Katsuhisa, Erik D. Demaine, Takehiro Ito, Jun Kawahara, Masashi Kiyomi, Yoshio Okamoto, Toshiki Saitoh, Akira Suzuki, Kei Uchizawa, and Takeaki Uno. "Swapping labeled tokens on graphs." Theoretical Computer Science 586 (2015): 81-94.

[7] Biniaz, Ahmad, Kshitij Jain, Anna Lubiw, Zuzana Masárová, Tillmann Miltzow, Debajyoti Mondal, Anurag Murty Naredla, Josef Tkadlec, and Alexi Turcotte. "Token swapping on trees." Discrete Mathematics & Theoretical Computer Science 24, no. Discrete Algorithms (2023).

[8] Vaughan, Theresa P. "Factoring a permutation on a broom." Journal of Combinatorial Mathematics and Combinatorial Computing 30 (1999): 129-148.

7.1 Source Code

7.1.1 Weighted Tokenswapping on Line

```
class Graph:
    def graph_input(self):
        n = int(input("Enter the number of vertices: "))
        graph = [[0, 0] for _ in range(n)]
        token_set = set()

        for i in range(n):
            token = int(input(f"Enter token at vertex {i + 1}: ")
                )

            if token > n:
                print(f"Error: Token value {token} exceeds the
                    number of vertices ({n}). "
                        "Please enter a value <= {n}.")
                return []

            if token in token_set:
                print(f"Error: Token value {token} is repeated.
                    Tokens must be unique.")
                return []

            token_set.add(token)

            weight = int(input(f"Enter weight of token {i + 1}: "
                ))
            graph[i][0] = token
            graph[i][1] = weight

        return graph


```

```python
def weighted_token_sort(graph):
    if not graph:
        return

    n = len(graph)
    swap_log = []
    total_cost = 0

    # Token-to-weight mapping
    weight_map = {token: weight for token, weight in graph}
    tokens = [token for token, _ in graph]

    # Process tokens from largest to smallest
    for target_token in sorted(tokens, reverse=True):
        current_pos = tokens.index(target_token)
        correct_pos = target_token - 1

        # Skip if already in correct place
        if current_pos == correct_pos:
            continue

        # Move token only to the right
        while current_pos < correct_pos:
            token_a = tokens[current_pos]
            token_b = tokens[current_pos + 1]
            cost = weight_map[token_a] + weight_map[token_b]
            swap_log.append((token_a, token_b, cost))
            total_cost += cost
            # Swap them
            tokens[current_pos], tokens[current_pos + 1] = (
                tokens[current_pos + 1],
                tokens[current_pos],
            )
            current_pos += 1
```

```
62
63      # Print swap log
64      for token_a, token_b, cost in swap_log:
65          print(f"Swap: {token_a} <-> {token_b}, Cost: {cost}")
66      print(f"Total cost: {total_cost}")
67
68
69  # Driver Code
70  if __name__ == "__main__":
71      my_graph = Graph()
72      graph_data = my_graph.graph_input()
73      weighted_token_sort(graph_data)
```

Listing 1: Weighted Tokenswapping on Line

7.1.2 Weighted Tokenswapping on Star

```
1   class StarTokenSwapper:
2       def __init__(self, n):
3           self.n = n
4           self.tokens = [0] * n
5           self.weights = {}
6           self.read_input()
7
8       def read_input(self):
9           seen = set()
10          for i in range(self.n):
11              token = int(input(f"Enter token at vertex {i+1}: "))
12              if token < 1 or token > self.n:
13                  raise ValueError(f"Token must be between 1 and {
                        self.n}")
14              if token in seen:
15                  raise ValueError(f"Duplicate token {token} found"
                        )
```

```
16              seen.add(token)
17
18              weight = int(input(f"Enter weight of token {token}: "
                    ))
19              self.tokens[i] = token
20              self.weights[token] = weight
21
22          print("\nInitial state:")
23          for i in range(self.n):
24              t = self.tokens[i]
25              print(f"  Vertex {i+1}: Token {t}, Weight {self.
                    weights[t]}")
26
27      def is_sorted(self):
28          return all(self.tokens[i] == i + 1 for i in range(self.n)
                )
29
30      def find_cycles(self):
31          visited = [False] * self.n
32          cycles = []
33          for i in range(self.n):
34              if not visited[i]:
35                  cycle = []
36                  cur = i
37                  while not visited[cur]:
38                      visited[cur] = True
39                      cycle.append(cur)
40                      cur = self.tokens[cur] - 1
41                  if len(cycle) > 1:
42                      cycles.append(cycle)
43          return cycles
44
45      def unlocked_cycle(self):
46          visited, cycle, cur = [False]*self.n, [], 0
```

```
47          while not visited[cur]:
48              visited[cur] = True
49              cycle.append(cur)
50              cur = self.tokens[cur] - 1
51          return cycle
52
53      def locked_cycles(self):
54          unlocked = set(self.unlocked_cycle())
55          return [c for c in self.find_cycles() if not set(c).
                issubset(unlocked)]
56
57      def find_x(self):
58          cycle = self.unlocked_cycle()
59          x = min(cycle, key=lambda v: self.weights[self.tokens[v
                ]])
60          return self.tokens[x], self.weights[self.tokens[x]]
61
62      def find_a(self):
63          min_token, min_w = None, float("inf")
64          for cyc in self.locked_cycles():
65              for v in cyc:
66                  t = self.tokens[v]
67                  if v != t-1 and self.weights[t] < min_w:  #
                        misplaced & lighter
68                      min_token, min_w = t, self.weights[t]
69          return (min_token, min_w) if min_token else (None, None)
70
71      def find_h(self):
72          min_token, min_w = None, float("inf")
73          for i in range(1, self.n):
74              if self.tokens[i] == i+1:
75                  t, w = self.tokens[i], self.weights[self.tokens[i
                        ]]
76                  if w < min_w:
```

```
                    min_token, min_w = t, w
        return (min_token, min_w) if min_token else (None, None)

    def choose_strategy(self):
        x, wx = self.find_x()
        a, wa = self.find_a()
        h, wh = self.find_h()

        if a is None:
            return 1, (x, wx, a, wa, h, wh)
        if wx <= wa and (wh is None or wx < wh):
            return 1, (x, wx, a, wa, h, wh)
        if wa < (wh if wh else float("inf")):
            return 2, (x, wx, a, wa, h, wh)
        return 3, (x, wx, a, wa, h, wh)

    def swap(self, v):
        t0, tv = self.tokens[0], self.tokens[v]
        cost = self.weights[t0] + self.weights[tv]
        self.tokens[0], self.tokens[v] = tv, t0
        print(f"Swap center(Token {t0},W={self.weights[t0]}) "
              f"<-> Vertex {v+1}(Token {tv},W={self.weights[tv]})
                 , Cost={cost}")
        return cost

    def perform_strategy(self):
        total = 0
        strat, (x, wx, a, wa, h, wh) = self.choose_strategy()

        print(f"\nChosen strategy S{strat}: x={x} w={wx}, a={a} w
           ={wa}, h={h} w={wh}\n")

        def bring(token):
            nonlocal total
```

```
            if token and self.tokens[0] != token:
                total += self.swap(self.tokens.index(token))

        def resolve_lockeds():
            nonlocal total
            for cyc in self.locked_cycles():
                for v in cyc:
                    if v and self.tokens[v] != v+1:
                        total += self.swap(v)
                        break

        if strat == 1:
            bring(x);  resolve_lockeds()
        elif strat == 2:
            bring(x);  bring(a);  resolve_lockeds();  bring(a)
        else:  # strat 3
            bring(x);  bring(h);  resolve_lockeds();  bring(h)

        # final sweeps on unlocked cycle
        while not self.is_sorted():
            for v in self.unlocked_cycle():
                if v and self.tokens[v] != v+1:
                    total += self.swap(v)
                    break
            else:
                break

        # summary
        print("\nFinal state:")
        for i in range(self.n):
            print(f"  Vertex {i+1}: Token {self.tokens[i]}")
        print(f"\nTotal cost: {total}")

def main():
```

```
    try:
        n = int(input("Enter number of vertices: "))
        if n < 4:
            print("Star must have at least 4 vertices")
            return
        StarTokenSwapper(n).perform_strategy()
    except Exception as e:
        print("Error:", e)

if __name__ == "__main__":
    main()
```

Listing 2: Weighted Token Swapping on Star

7.1.3 Tokenswapping on Single Broom

```
class SingleBroom:
    def __init__(self, k, pi):
        self.k = k
        self.original = pi
        self.pi = pi.copy()
        self.n = len(pi)
        self.swaps = []


    def is_sorted(self):
        return all(self.pi[i] == i + 1 for i in range(self.n))

    def star_swap(self):
        n = self.k
        while self.pi[:n] != sorted(self.pi[:n]):
            i = self.pi[n - 1]
            if i != n:
                tgt = i - 1
```

```
19                  if tgt < n - 1:
20                      self.pi[n-1], self.pi[tgt] = self.pi[tgt],
                            self.pi[n-1]
21                      self.swaps.append((self.pi[n-1], self.pi[tgt
                            ]))
22              else:
23                  for j in range(n - 1):
24                      if self.pi[j] != j + 1:
25                          i = self.pi[j]
26                          tgt = i - 1
27                          self.pi[n-1], self.pi[tgt] = self.pi[tgt
                                ], self.pi[n-1]
28                          self.swaps.append((self.pi[n-1], self.pi[
                                tgt]))
29                          break
30
31      def s_min(self):
32          k, n = self.k, self.n
33          star = self.pi[:k-1]
34          path = self.pi[k-1:n]
35
36          p = 0
37          while p < len(path):
38              smin = path[p]
39              if 0 < smin < k and smin <= len(star):
40                  while p:
41                      path[p], path[p-1] = path[p-1], path[p]
42                      self.swaps.append((path[p], path[p-1]))
43                      p -= 1
44                  self.swaps.append((star[smin-1], path[0]))
45                  star[smin-1], path[0] = path[0], star[smin-1]
46                  p = 0
47              else:
48                  p += 1
```

```
49          self.pi = star + path
50
51      def p_max(self):
52          k = self.k
53          path = self.pi[k-1:]
54          while path:
55              pmax = max(path)
56              if pmax == k:
57                  path.remove(pmax)
58                  continue
59              cur = self.pi.index(pmax)
60              tgt = pmax - 1
61              while cur != tgt:
62                  nxt = cur + 1 if cur < tgt else cur - 1
63                  self.swaps.append((self.pi[cur], self.pi[nxt]))
64                  self.pi[cur], self.pi[nxt] = self.pi[nxt], self.
                        pi[cur]
65                  cur = nxt
66              path.remove(pmax)
67
68      def sort(self):
69          if self.is_sorted():
70              print("Permutation is already sorted."); return
71
72          self.s_min()
73          print("After s_min:", self.swaps)
74
75          if self.is_sorted():
76              print("Sorted!\nInitial:", self.original, "\nFinal:",
                    self.pi); return
77
78          self.p_max()
79          print("After p_max:", self.swaps)
80
```

```
        if self.is_sorted():
            print("Sorted!\nInitial:", self.original, "\nFinal:",
                self.pi); return

        self.star_swap()
        print("After star swaps:", self.swaps)

        print("Sorted!\nInitial:", self.original, "\nFinal:",
            self.pi)

def main():
    try:
        n = int(input("Number of vertices: "))
        if n < 4:
            print("Need at least 4 vertices.")
            return
        k = int(input("Size of star (k): "))
        pi = list(map(int, input("Enter permutation separated by
            spaces: ").split()))
        swapper = SingleBroom(k, pi)
        swapper.sort()
    except Exception as e:
        print("Error:", e)

if __name__ == "__main__":
    main()
```

Listing 3: SingleBroom: Three-phase Sorting on a Broom